Rapport Codage de Huffman (GH03)

Abel Fares, Yan Rabefaniraka

11 Janvier 2025

Contents

1 Résumé **2**

2 Introduction **3**

3 Modules de l'application **4**
3.1 Schéma de l'application . . . 4
3.2 Structures de données . . . 5
3.3 Compression . . . 5
3.3.1 Fonctionnement . . . 5
3.3.2 Principaux algorithmes . . . 6
3.3.3 Structures de données utilisées . . . 8
3.4 Décompression . . . 8
3.4.1 Fonctionnement . . . 8
3.4.2 Principaux algorithmes . . . 9
3.4.3 Structures de données utilisées . . . 11

4 Tests réalisés **12**
4.1 Arbre . . . 12
4.2 Huffman . . . 12
4.3 Exemple du sujet . . . 12
4.3.1 Résultat (*xxd*) . . . 12
4.4 Fichier vide . . . 14
4.5 Mauvaise extension . . . 14
4.6 Tests d'arguments . . . 14
4.7 Long fichier . . . 14
4.7.1 Résultat (sans *xxd*, ce serait trop long) . . . 14

5 Organisation du travail **16**
5.1 Tableau `Qui (a) fait Quoi` . . . 16

6 Difficultés rencontrées **18**
6.1 Choix des structures de données . . . 18
6.2 Lier les parties du sujet ensemble . . . 18
6.3 Manipuler la modularité de Ada et savoir où et quand implémenter/instancier des structures de données. 19
6.4 Gérer notre temps et le répertoire . . . 19

7 Solutions **19**

8 État actuel du projet **19**
8.1 Avancée de Compression . . . 19
8.1.1 Valgrind . . . 19
8.2 Avancée de Décompression . . . 20
8.2.1 Valgrind . . . 20

8.3 Insuffisance(s) 20
8.3.1 Division des tâches 20
8.3.2 Fichiers binaires 20
8.4 Pistes d'amélioration 20

9 Bilans personnels et conclusion **21**

Remarque : Fichier engendré avec:

```
pandoc --toc -N -o ../livrables/rapport.pdf rapport_1.txt
```

1 Résumé

Ce rapport détaille notre réalisation du codage de Huffman en Ada – dont l'objectif et de compresser du texte (fichier binaire) pour en réduire la taille de stockage – mais aussi de la décompression d'un fichier qui a au préalable été compressé avec ce code. On y décrit notre compréhension du problème, notre approche, l'organisation du travail, nos difficultés ainsi que nos réussites. Enfin, on y apporte aussi un bilan personnel avec des précisions sur nos investissements personnels.

2 Introduction

Dans le cadre de l'unité d'enseignement de Programmation Impérative du Semestre 5, nous avons un projet "long" à réaliser en binôme sur une période d'environ 1 mois et demi. Nous avions pour consigne d'implémenter dans le langage modulaire Ada le codage de Huffman – un algorithme de compression – en appliquant tous les préceptes engrangés jusque là dans la matière.

Le codage de Huffman est un algorithme permettant de compresser des fichiers selon l'occurrence des symboles/octets dans la source. Ainsi, pour un fichier texte, le code est déterminé par le nombre de fois où apparaît un caractère/symbole. En particulier, plus un symbole est fréquent, plus son codage sera court. À l'inverse, un symbole peu récurrent aura un codage plus long. Avec cette méthode, on obtient une compression de données sans perte, *i.e* en décodant le fichier compressé, on est supposés retrouver les mêmes octets d'origine. Notre but a donc été d'implémenter ce codage en Ada. Pour ce faire, nous devions d'abord comprendre les détails de l'implémentation spécifique qui nous est demandée: On veut un programme capable de prendre le fichier en entrée, de trouver l'occurrence des symboles afin d'en générer un arbre de Huffman, puis d'utiliser les codes obtenus à l'aide de cet arbre pour coder le texte d'origine. Cela doit se faire sans prendre en compte le codage du fichier d'origine (ASCII, utf8, etc.). En particulier, notre programme inscrit dans le fichier de sortie les octets des symboles utilisés ainsi qu'un symbole de fin de texte permettant d'indiquer l'arrêt du décodage, suivis du parcours infixe permettant de générer l'arbre, puis enfin du texte codé. Ensuite, on doit faire un programme – ne prenant en entrée valide que les fichiers compressés par notre algorithme – permettant de décompresser ce dernier afin de retrouver l'exact même fichier en faisant le cheminement inverse.

Pour présenter notre approche, on se propose donc de développer les différents modules de notre application et leurs relations, tout en abordant nos principaux choix de conception. On plongera aussi en détail dans les structures de données et les algorithmes utilisés, nos tests réalisés et leur succès/échec, nos difficultés et solutions, l'organisation du travail, avec pour finir un bilan sur l'avancement du projet suivi de bilans personnels sur les ressentis et impacts du projet sur le binôme.

3 Modules de l'application

3.1 Schéma de l'application

Ci-dessous est la représentation de notre application.

```
[/src]
  |
   \Compression
        |
         \compression.adb
   \Decompression
        |
         \decompression.adb
   \Arbre
        |
         \arbre.ads
         \arbre.adb
   \LCA
        |
         \lca.ads
         \lca.adb
   \octets
        |
         \octet.ads
         \octet.adb
   \Sda_exceptions
        |
         \sda_exceptions.ads
   \[results]
```

- **Compression** : Ceci est le module principal de compression de l'application. C'est celui qu'on appelle dans le terminal pour compresser un fichier. Dedans, on fait appel à `Arbre`, `LCA`, `Octets` et `Sda_exceptions`. Cela nous donne accès aux différentes structures de données dont on a besoin avec des procédures pour les manipuler, ainsi qu'à une gestion des exceptions. Il ouvre un nouveau fichier dans lequel il écrit les octets originaux des symboles codés, le parcours infixe de l'arbre de Huffman, suivi du texte codé à l'aide de ce dernier.
- **Decompression** : Ceci est le module de decompression de l'application. Il fait appel aux mêmes modules que `Compression` excepté LCA. On trouve notamment dans ces derniers les procédures pour recréer un Arbre de Huffman à partir d'un parcours infixe ou encore celui pour décoder du texte à partir de ce même arbre. Ainsi, ce module récupère les symboles en début de fichier, puis récupère l'arbre à partir du parcours infixe, et enfin décode le reste du fichier à l'aide de l'arbre.
- **Arbre** : Dans ce module générique, on définit notre structure de données d'Arbre de Huffman ainsi que les procédures qui en découlent. On y définit aussi les différentes listes chaînées utilisées. Dans ces procédures, on a notamment les différentes procédures d'affichage de l'arbre et des listes, la procédure pour récupérer l'encodage binaire d'un symbole dans l'arbre de Huffman, retrouver l'arbre à partir du parcours infixe, ou encore celui pour décoder du texte codé à l'aide de l'arbre.
- **LCA** : Ce module est utilisé par `Compresser` pour stocker une liste chaînée associative des symboles et de leur fréquence. Se référer au *pr-02*.
- **Octet** : Ce module définit le type de données `T_Octet` qui est un entier modulaire, accompagné d'un pointeur vers une liste d'octets. On les utilise dans les deux modules principaux pour conserver et manipuler nos octets. Il s'y trouve en particulier la procédure pour récupérer des octets dans un fichier binaire.
- **Sda_exceptions** : Dans cet en-tête sont définies nos exceptions.
- [**results**] : Ce dossier stocke les résultats. À l'heure de l'écriture de ce rapport, on planifie d'y écrire les fichiers de tests après utilisation d'une commande ***Bash***.

3.2 Structures de données

3.2.0.1 Listes chaînées associatives

- Les LCA constituent la plupart des tableaux qu'on utilise, que ce soit pour enregistrer les parcours infixe ou les symboles et leur occurrence. On utilise cette structure pour éviter d'y assigner une taille arbitraire, tout en assurant une bonne gestion de la mémoire.

3.2.0.2 Arbre Binaire

- Cette structure est constituée d'un pointeur qui donne accès à un enregistrement où on associe une clé d'un type que l'on a défini dans compression comme `T_String` : un string de taille 2, une valeur de type défini comme entier dans compression, ainsi qu'un fils gauche et droit qui sont eux-mêmes des pointeurs vers des arbres.

3.2.0.3 Table d'arbres

- Cette structure est un pointeur qui donne accès a un enregistrement d'un arbre et d'un pointeur qui donne accès a la cellule suivante.

3.2.0.4 Table infixe

- Cette structure est une liste chaînée d'entiers c'est-à-dire un pointeur donnant accès à un enregistrement regroupant un entier et le pointeur permettant d'accéder à la cellule suivante.

3.2.0.5 Table symbole

- C'est une liste chaînée de symboles de type `T_Cle`.

3.2.0.6 Table de Huffman

- C'est un pointeur donnant accès à un enregistrement regroupant un symbole de type `T_Cle`, un tableau d'encodage de type `T_ptr_infixe`, et un pointeur donnant accès à la prochaine cellule.

3.2.0.7 Type Octet

- Cette structure est un entier modulaire sur 2^8. Elle nous permet de stocker nos octets sur 256 valeurs, de les manipuler comme tel et de les afficher comme des entiers. On peut en récupérer le bit de poids fort en utilisant la méthode de Horner: On divise l'octet par 128 pour récupérer le bit, puis on multiplie l'octet par deux pour 'décaler' ses bits vers la gauche. Pour explorer un octet, on peut ainsi avoir un indice de bits qu'on incrémente jusqu'à 8.

3.2.0.8 Tableau d'octet

- En utilisant un pointeur vers un tableau d'octets, on peut stocker les octets lus dans un tableau et ainsi y accéder facilement avec un index à incrémenter. Cet index augmente de 1 à chaque fois que l'indice des bits atteint 8. À préciser que notre utilisation de cette structure est donc destructive puisqu'on modifie l'octet courant au plus 8 fois.

3.3 Compression

3.3.1 Fonctionnement

Notre programme de compression est basé sur le codage de Huffman. On met en argument de l'appel du programme les options d'affichage `-s` / `--silencieux` pour silencieux et `-b` / `--bavard` pour bavard (par défaut) pour un fichier texte qu'on veut compresser:

```
./compression  -s -b texte.txt
```

Le programme va récupérer chaque symbole utilisé, et stocker dans un tableau le symbole et sa fréquence associée dans le texte. Pour ce faire, on utilise une liste chaînée associative qui nous permet d'associer une clé à une valeur,

ici un symbole à sa fréquence. On ajoute également un symbole de fin '**\$**' de fréquence 0. Ce dernier permettra à la décompression de savoir que le fichier est terminé et qu'il peut s'arrêter de lire.

Ce tableau d'occurrences permet ensuite de créer un arbre binaire dont les feuilles sont les noeuds contenant les symboles et leur fréquence associée et ayant leurs fils gauches et droits définis comme le pointeur *null*. On a choisi de stocker ces arbres dans un tableau d'arbre qui est en fait une structure chaînée d'arbres.

A partir de ce tableau, on trouve les 2 noeuds à occurrences minimales afin d'en constituer le noeud parent dont la valeur associée est la somme des occurrences de ses fils puis on supprime les noeuds fils gardant uniquement le nouveau noeud. Le noeud à la plus grande valeur est placé en fils droit et l'autre en fils gauche. Si le mode bavard est activé, l'arbre ainsi que la table de Huffman associée seront affichés après la fin de cette opération.

Ensuite, on génère un tableau où on écrit le parcours infixe de l'arbre; 0 pour l'avancement à gauche, 1 pour l'arrivée à une feuille. On génère également un tableau des symboles dans ce même ordre infixe. On en aura besoin pour renvoyer les codages binaires des dits symboles.

On peut affichera alors l'arbre de la manière suivante:

```
(4)
\--0--(2)
|      \--0--(1)'l'
|      \--1--(1)'x'
\--1--(2)
```

On va finalement générer la table de Huffman après le filtrage de toutes ces données, qui est composée de chaque symbole et de leur encodage binaire associé et ce dans l'ordre des symboles dans l'arbre de Huffman. On va aussi générer la table des entiers associés aux symboles en ASCII en plaçant comme premier élément la position du symbole de fin dans la liste de symbole puis en doublant le dernier élément pour qu'on sache quand s'arrête la liste des symboles lors de la décompression.

Enfin, on peut écrire notre fichier compressé au format `.hff`. On y met, à l'aide de la méthode de Horner pour écrire les octets:

```
1. La liste des symboles dans l'ordre du parcours infixe en binaire.
2. Le parcours infixe de l'arbre
3. Le contenu du texte sous sa forme encodée sans oublier le symbole de fin
```

3.3.2 Principaux algorithmes

3.3.2.1 Générer l'arbre de Huffman grâce aux occurrences

- Dans `Compression.adb` :

```
-- Créer l'arbre de Huffman
   while Taille_Table_Arbre(Table_arbres) > 1 loop
      Min(Table_arbres, Min1, Min2);
      if AB_La_Valeur(Min1) > AB_La_Valeur(Min2) then
         AB_Creer_Noeud(Nouvel_Arbre,To_T_String('0'),
         AB_La_Valeur(Min1) + AB_La_Valeur(Min2), Min2, Min1);
      else
         AB_Creer_Noeud(Nouvel_Arbre,To_T_String('0'),
         AB_La_Valeur(Min1) + AB_La_Valeur(Min2), Min1, Min2);
      end if;
      Enregistrer_Arbre(Table_arbres, Nouvel_Arbre);
   end loop;
```

3.3.2.2 Créer les tableaux de parcours infixe et de symbole

- Dans `Arbre.adb` :

```
procedure Table_symbole_infixe(Arbre : in T_ARBRE;
Tab_symbole : out T_ptr_symbole;
Tab_arbre : out T_ptr_infixe) is
```

```
   begin
      if Arbre.all.Fils_Gauche /= Null then
         Enregistrer_infixe(Tab_arbre, 0);
         Table_symbole_infixe(Arbre.all.Fils_Gauche,
         Tab_symbole, Tab_arbre);
         Table_symbole_infixe(Arbre.all.Fils_Droit,
         Tab_symbole, Tab_arbre);
      else
         Enregistrer_infixe(Tab_arbre, 1);
         Enregistrer_symbole(Tab_symbole, Arbre.all.Cle);
      end if;
   end Table_symbole_infixe;
```

3.3.2.3 Récupérer le codage d'un symbole dans l'arbre

- Dans `Arbre.adb` :

```
procedure Symbole_encodage(Arbre : in T_ARBRE; Symbole : in T_Cle;
gauche : in Boolean; indice : in Integer;
Tab : in out T_ptr_infixe; Tab_out : out T_ptr_infixe)
is
   Curseur : T_ptr_infixe :=Tab;
   begin
      if indice=0 then
         Symbole_encodage(Arbre.all.Fils_Gauche, Symbole,
         True, indice+1, Tab, Tab_out);

         Symbole_encodage(Arbre.all.Fils_Droit, Symbole,
         False, indice+1, Tab, Tab_out);
      else
         --Effacer les valeurs de Tab ayant un indice supérieur
         -- à l'indice actuel
         if Tab/=Null then
            if indice = 1 and Tab.all.Suivante /= Null then
               Detruire_infixe(Tab.all.Suivante);
               Tab.all.Suivante := Null;
            elsif indice > 1 then
               for i in 1..indice-1 loop
                  Curseur:=Curseur.all.Suivante;
               end loop;
               if Curseur/=Null then
                  Detruire_infixe(Curseur.all.Suivante);
               end if;
            end if;
         end if;
         if gauche then
            Enregistrer_infixe_indice(Tab,0,indice);
         else
            Enregistrer_infixe_indice(Tab,1,indice);
         end if;
         if AB_Est_Feuille(Arbre) then
            if Arbre.all.Cle=Symbole then
               Assigner_Infixe(Tab, Tab_out);
            end if;
         else
            Symbole_encodage(Arbre.all.Fils_Gauche, Symbole,
            True, indice+1, Tab, Tab_out);
```

```
            Symbole_encodage(Arbre.all.Fils_Droit, Symbole,
            False, indice+1, Tab, Tab_out);
         end if;
      end if;
   end Symbole_encodage;
```

3.3.3 Structures de données utilisées

Listes chaînées associatives :On stocke le tableau à double entrée des fréquences de symbole dans une liste chaînée associative.

Arbre Binaire : La structure de l'arbre binaire est utile pour générer notre arbre de Huffman (parfait). On emploie notamment sa fonction permettant de récupérer le codage d'un symbole, et ce afin de compresser le texte d'origine.

Table d'arbres : Le Tableau d'arbres est utilisé dans la création de l'arbre de Huffman en stockant toutes les feuilles qu'on génère au départ du programme. Elle évolue au fil du programme avec la suppression des noeuds à valeur minimale.

Table infixe : Cette structure originellement utilisée pour stocker l'arbre de Huffman sous la forme de son parcours infixe en binaire a finalement servi pour stocker l'encodage binaire des symboles, résultat du parcours de l'arbre atteignant la feuille du symbole, mais aussi la liste des symboles dans l'ordre de l'arbre de Huffman sous la forme de l'entier qui les encode en ASCII.

Table symbole : Cette structure permet d'enregistrer les symboles dans l'ordre défini par l'arbre de Huffman.

Table de Huffman : Cette structure est utilisée pour stocker l'association des symboles et de leur encodage binaire, le tout dans l'ordre de l'arbre de Huffman.

T_Octet : Ici, on utilise uniquement le type Octet pour sa modularité afin d'employer la méthode de Horner dans l'écriture du fichier binaire.

3.4 Décompression

3.4.1 Fonctionnement

`Decompression` a les mêmes arguments d'affichage que `Compression` avec `-s` / `--silencieux` pour silencieux et `-b` / `--bavard` pour bavard (par défaut). Pour décompresser un fichier, on procède comme suit:

```
./decompression -s -b texte_compresse.txt.hff
```

Seuls les fichiers dotés de l'extension `.hff` peuvent être ouverts, sans quoi une erreur est renvoyée.

Une fois le fichier ouvert, le programme récupère tous les octets du fichier en entrée dans un tableau d'octets. On peut ensuite récupérer les symboles du texte d'origine dans un tableau chaîné de symboles, en bouclant dans le tableau d'octets; Pour cela, on a un pointeur principal dont l'octet pointé sera enregistré, et un pointeur en avance d'un octet pour vérifier la répétition de l'octet courant. On pourra donc arrêter d'enregistrer les octets une fois que l'octet courant se répète une deuxième fois.

Ensuite, on utilise une procédure récursive pour récupérer l'arbre de Huffman à partir du parcours infixe qui suit le doublon d'octet: Cette fonction si elle rencontre un *0*, crée un fils gauche puis relance la récursion à l'intérieur. Si *1* est rencontré, cela veut dire qu'on arrive sur une feuille: on peut donc y inscrire le symbole courant et arrêter la récursion dans ce sous-arbre. Puis, on fait de même pour le fils droit. Une fois que tous les symboles sont inscrits dans une feuille, la procédure s'arrête. On aura reproduit l'arbre de Huffman obtenu par le codage de Huffman. Accessoirement, on lit le bit courant en utilisant la méthode de *Horner* ($bit := octet/128$ pour récupérer le bit de poids fort puis $octet := octet * 2$ pour décaler les bits à gauche.)

Et comme nos différents indices d'exploration (indice du bit courant, indice de l'octet courant) sont en mode `In Out`, on peut continuer d'explorer les bits suivants pour maintenant décoder le texte compressé à l'aide de l'arbre et d'une autre procédure récursive permettant d'y récupérer un symbole. En effet, en lisant 0 (*resp.* 1) sur le bit courant, on descend dans le fils gauche (*resp.* droit) du noeud courant. Une fois la feuille atteinte, on renvoie le symbole qu'elle a en valeur. On répète ainsi jusqu'à trouver le symbole de fin de texte.

Durant son fonctionnement, si en mode *bavard*, le module peut par exemple afficher les étapes suivantes:

```
Position du caractère de fin :  0
```

```
Octets récupérés :  0; 101; 100; 97; 99; 98; 98; 29; 96; 190; 72;

Tableau des symboles :
 '\$' 'e' 'd' 'a' 'c' 'b'

Arbre d'Huffman :
( 0)
\--0--( 0)
|     \--0--( 0)
|     |     \--0--( 0) '\$'
|     |     \--1--( 0) 'e'
|     \--1--( 0) 'd'
\--1--( 0)
      \--0--( 0) 'a'
      \--1--( 0)
            \--0--( 0) 'c'
            \--1--( 0) 'b'
Index d'exploration et bit courants: 9;  4
Symboles décodés :
 'a' 'b' 'c' 'd' 'e' '\$'

Texte décompressé :
abcde
```

Enfin, comme pour `Compresser`, on écrit notre fichier décompressé au format `.hff.d`. Seul problème: dans l'état actuel de notre application, on écrit dans un fichier texte au lieu de réécrire les octets de nos symboles.

3.4.2 Principaux algorithmes

À partir des sous-programmes écrits avec `Compresser`, il a été un peu plus aisé de réaliser `Décompresser`. De même, les deux principaux sous-programmes *challengeant* étant la génération de l'arbre et le décodage du texte, le fait que leur récursivité utilise le même principe hérité du parcours infixe (côté gauche –> noeud –> côté droit) a donné des algorithmes plutôt similaires dans leur structures et donc plus abordables à réaliser.

3.4.2.1 Récupérer le tableau des symboles/octets

- Dans `decompression.adb`:

```
--Créer Tableau de symboles (on les gardera en binaire
--dans la version finale)
   Init_symbole(Tab_Symbole_Binaire_Original);

   --Ajouter les symboles avant le caractère de fin
   for i in 2..Position_char_fin+1 loop
      Octet := Tab_Entree.all(File_Size(i));
      if Octet = 10 then
         Enregistrer_symbole(Tab_Symbole_Binaire_Original, "\n");
      else
         Tmp_Char := Character'Val(Octet);
         Enregistrer_symbole(Tab_Symbole_Binaire_Original,
         To_T_String(Tmp_Char));
      end if;
   end loop;

   --Ajouter le symbole de fin
   Tmp_String := ('\', '$');
   Enregistrer_symbole(Tab_Symbole_Binaire_Original, Tmp_String);
   Tmp_Exploration_Index := Position_char_fin + 2;
```

```
   Tmp_Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index+1));

   --Ajouter les symboles après le caractère de fin
   -- jusqu'à répétition d'un octet.
   Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index));
   while not Est_Meme_Octet(Octet, Tmp_Octet) loop
      if Octet = 10 then
         Enregistrer_symbole(Tab_Symbole_Binaire_Original, "\n");
      else
         Tmp_Char := Character'Val(Octet);
         Enregistrer_symbole(Tab_Symbole_Binaire_Original,
         To_T_String(Tmp_Char));
      end if;
      Tmp_Exploration_Index := Tmp_Exploration_Index + 1;
      Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index));
      Tmp_Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index+1));
   end loop;
   Tmp_Char := Character'Val(Octet);
   Enregistrer_symbole(Tab_Symbole_Binaire_Original, To_T_String(Tmp_Char));
```

3.4.2.2 Générer l'arbre avec le parcours infixe

- La procédure `Generer_Arbre_Parcours` dans `arbre.adb`:

```
--Générer l'arbre de Huffman à partir du parcours infixe
   procedure Generer_Arbre_Parcours(Tab_symbole : in T_ptr_symbole;
   Tab_Entree : in T_ptr_Tab_Octet;
   Arbre, Arbre_vide : in out T_ARBRE; Indice_Symboles, Indice_Bit,
   Indice_Parcours_Entree, Indice_Symboles_Max : in out Integer;
   Cle_Default : in T_Cle; Valeur_Default: in T_Valeur) is
      Bit : T_Octet;
      Octet : T_Octet := Tab_Entree.all(File_Size(Indice_Parcours_Entree));
   begin
      Bit := Octet / 128;
      Tab_Entree.all(File_Size(Indice_Parcours_Entree)) := Octet*2;
      if Indice_Symboles <= Indice_Symboles_Max then
         if Indice_Bit = 8 then
               Indice_Bit := 1;
               Indice_Parcours_Entree := Indice_Parcours_Entree + 1;
            else
               Indice_Bit := Indice_Bit + 1;
            end if;
         if Bit = 1 then
            AB_Assigner_Cle(Arbre, Symbole_Indice(Tab_symbole,
            Indice_Symboles));

            Indice_Symboles := Indice_Symboles + 1;
         else
            AB_Creer_Noeud(Arbre.all.Fils_Gauche, Cle_Default,
            Valeur_Default, Arbre_vide, Arbre_vide);

            AB_Creer_Noeud(Arbre.all.Fils_Droit, Cle_Default,
            Valeur_Default, Arbre_vide, Arbre_vide);

            Generer_Arbre_Parcours(Tab_symbole, Tab_Entree,
            Arbre.all.Fils_Gauche,
            Arbre_vide, Indice_Symboles, Indice_Bit,
```

```
            Indice_Parcours_Entree, Indice_Symboles_Max,
            Cle_Default, Valeur_Default);

            Generer_Arbre_Parcours(Tab_symbole, Tab_Entree,
            Arbre.all.Fils_Droit,
            Arbre_vide, Indice_Symboles, Indice_Bit,
            Indice_Parcours_Entree, Indice_Symboles_Max,
            Cle_Default, Valeur_Default);
         end if;
      end if;
   end Generer_Arbre_Parcours;
```

3.4.2.3 Décoder le texte avec l'arbre

- La fonction `Decoder_Symbole_Compresse` dans `arbre.adb`:

```
--Décoder un symbole compressé à partir de l'arbre de Huffman
   function Decoder_Symbole_Compresse(Arbre: in T_ARBRE;
   Tab_Entree : in T_ptr_Tab_Octet; Indice_Bit,
   Indice_Parcours_Entree : in out Integer) return T_Cle is
      Bit : T_Octet;
      Octet : T_Octet := Tab_Entree.all(File_Size(Indice_Parcours_Entree));
   begin
      if AB_Est_Feuille(Arbre) then
         return AB_La_Cle(Arbre);
      else
         Bit := Octet / 128;
         Tab_Entree.all(File_Size(Indice_Parcours_Entree)) := Octet*2;
         if Indice_Bit = 8 then
            Indice_Bit := 1;
            Indice_Parcours_Entree := Indice_Parcours_Entree + 1;
         else
            Indice_Bit := Indice_Bit + 1;
         end if;
         if Bit = 1 then
            return Decoder_Symbole_Compresse(Arbre.all.Fils_Droit,
            Tab_Entree, Indice_Bit, Indice_Parcours_Entree);
         else
            return Decoder_Symbole_Compresse(Arbre.all.Fils_Gauche,
            Tab_Entree, Indice_Bit, Indice_Parcours_Entree);
         end if;
      end if;
   end Decoder_Symbole_Compresse;
```

3.4.3 Structures de données utilisées

Arbre Binaire : La structure de l'arbre binaire permet de retrouver le codage de nos symboles à l'aide du parcours infixe de l'arbre de Huffman.

Type Octet : Ce type modulaire est utilisé pour stocker chaque octet du fichier à décompresser. On les manipule ensuite dans les fonctions de génération d'arbre et de décodage de texte compressé à l'aide de la méthode de Horner.

Tableau d'octet : On y stocke tous les octets du fichier en entrée. Cela rend l'accès simplifié.

(TEMPORAIRE) Table symbole : On utilise la table de symbole simplement pour stocker les symboles ASCII associés aux octets récupérés. Cela permet leur affichage. Cependant, cela ne prend pas en compte les symboles pouvant être codés sur plusieurs bits et dans d'autres codages. C'est un des détails à revoir si on veut améliorer le projet et le rendre plus conforme au cahier des charges.

4 Tests réalisés

4.1 Arbre

En préambule, on a effectué un test minimal de la seule nouvelle structure de donnée qu'est l'arbre de Huffman; On a ainsi effectué des tests unitaires pour toutes les procédures dans le fichier `test_arbre.adb`.

4.2 Huffman

Notre objectif avec ces tests est d'évaluer:

- Le bon fonctionnement des arguments *i.e* la prise en compte des options *bavard* et *silencieux* ainsi que l'extension du fichier d'entrée.
- Les cas limites tels que les symboles spéciaux (codage à nombre d'octets variables selon le codage du texte) ou le fichier vide.
- La restitution totale du fichier d'origine (le fichier est compressé sans perte et la décompression renvoie l'exacte copie de l'originel).

Il semble également important de préciser que pour l'instant, `Compression` ne prend en compte que des fichiers textuels codés en ASCII. Ainsi, on a préféré orienter nos tests vers ce codage de fichier.

4.3 Exemple du sujet

Le fichier `texte_sujet.txt` est un exemple simple. On y trouve des symboles communs ainsi que des symboles spéciaux comme ':' ou encore un *line feed* (`\n`) et un *carriage return* (`\r`). L'occurrence distinctement élevée du caractère 'e' permet de vérifier qu'il obtient bien le codage le plus petit à l'inverse du symbole de fin ou du symbole de *line feed*.

```
exemple de texte :
exempte tempete lexeme
```

4.3.1 Résultat (*xxd*)

Avec le mode *bavard*, on observe la sortie suivante pour la compression:

La sortie en fichier binaire est:

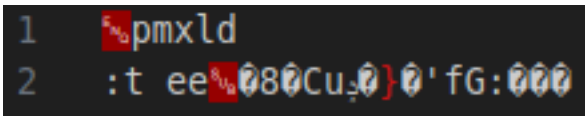


Figure 1: texte_sujet.txt.hff

On peut explorer les bits avec la commande suivante:

```
xxd -b texte_sujet.txt.hff
```

On obtient:

```
00000000: 00000101 01110000 01101101 01111000 01101100 01100100  .pmxld
00000006: 00001010 00111010 01110100 00100000 01100101 01100101  .:t ee
0000000c: 00011010 10100111 00111000 11010110 01000011 01110101  ..8.Cu
00000012: 11011101 10011010 10011101 01111101 11101110 10110010  ...}..
00000018: 00100111 01100110 01000111 00111010 11011010 11001110  'fG:..
0000001e: 11110000
```

Le résultat semble être concluant puisqu'on retrouve en premier octet la position de notre symbole de fin, nos symboles originels, ainsi que la répétition du dernier symbole.

Maintenant, avec `Décompresser` en mode *bavard*, on obtient bien:

```
Position du caractère de fin :  5

Octets récupérés :  5; 112; 109; 120; 108; 100; 10; 58; 116; 32;
101; 101; 26; 167; 56; 214; 67; 117; 221; 154; 157; 125; 238; 178;
39; 102; 71; 58; 218; 206; 240;

Symboles précédant le caractère de fin :
 'p' 'm' 'x' 'l' 'd'

Tableau des symboles :
 'p' 'm' 'x' 'l' 'd' '\$' '\n' ':' 't' ' ' 'e'

Arbre d'Huffman :
( 0)
\--0--( 0)
|     \--0--( 0)
|     |     \--0--( 0) 'p'
|     |     \--1--( 0) 'm'
|     \--1--( 0)
|           \--0--( 0) 'x'
|           \--1--( 0)
|                 \--0--( 0) 'l'
|                 \--1--( 0)
|                       \--0--( 0) 'd'
|                       \--1--( 0)
|                             \--0--( 0)
|                             |     \--0--( 0) '\$'
|                             |     \--1--( 0) '\n'
|                             \--1--( 0) ':'
\--1--( 0)
      \--0--( 0)
      |     \--0--( 0) 't'
      |     \--1--( 0) ' '
      \--1--( 0) 'e'
Index d'exploration et bit courants: 15;  6

Texte décompressé :
exemple de texte :
exempte tempete lexeme
```

On peut facilement vérifier l'égalité des fichiers avec la commande suivante:

```
diff texte_sujet.txt /results/texte_sujet.txt.hff.d
```

À l'écriture du rapport, on a le résultat ci-dessous:

```
2c2
< exempte tempete lexeme
\ Pas de fin de ligne à la fin du fichier
---
> exempte tempete lexeme
```

On peut en déduire que le fichier décode correctement le texte compressé, cependant il semble ajouter un *line feed* en trop. Cela vient probablement de la fonction d'écriture du package `Unbounded_IO` qui ajoute automatiquement un saut de ligne à la fin de l'écriture.

4.4 Fichier vide

En lançant le fichier vide, `Compression` lève une erreur. En effet, sans octet à récupérer, le programme ne peut fonctionner. Ce n'est pas un cas auquel nous avons réfléchi dans notre programmation défensive, mais qu'on pourrait totalement intégrer à nos pré-conditions.

4.5 Mauvaise extension

`Compression` n'a pas de contrainte sur l'extension du fichier en entrée, à l'inverse de `Decompression`.

Le fichier `mauvaise_extension.hfd`, bien que binaire, n'a pas l'extension *.hff*. Voici ce que retourne `Decompression`:

```
Erreur: Le fichier n'est pas un fichier compressé de Huffman.
Veuillez vérifier vos arguments.
```

C'est bien le résultat attendu.

4.6 Tests d'arguments

Notre programme est par défaut en mode bavard. Cependant, avec les arguments `-b` / `--bavard` et `-s` / `--silencieux` on peut préciser l'affichage souhaité. En particulier, seul le dernier argument est supposé être pris en compte par le programme. On teste cela par exemple avec la commande:

```
./compression -b -s -b -s texte_sujet.txt
```

et

```
./decompression -b -s -b --silencieux results/texte_sujet.txt.hff
```

On peut remarquer en lançant ces commandes que les arguments sont pris en compte comme on le souhaite par les deux modules.

De même, les arguments invalides renvoient l'erreur qui suit:

```
Utilisation: ./decompression [-b | --bavard | -s | --silencieux] <file_name>.
Veuillez vérifier vos arguments.
```

4.7 Long fichier

Enfin, `texte_long.txt` est le texte *lorem ipsum* suivant:

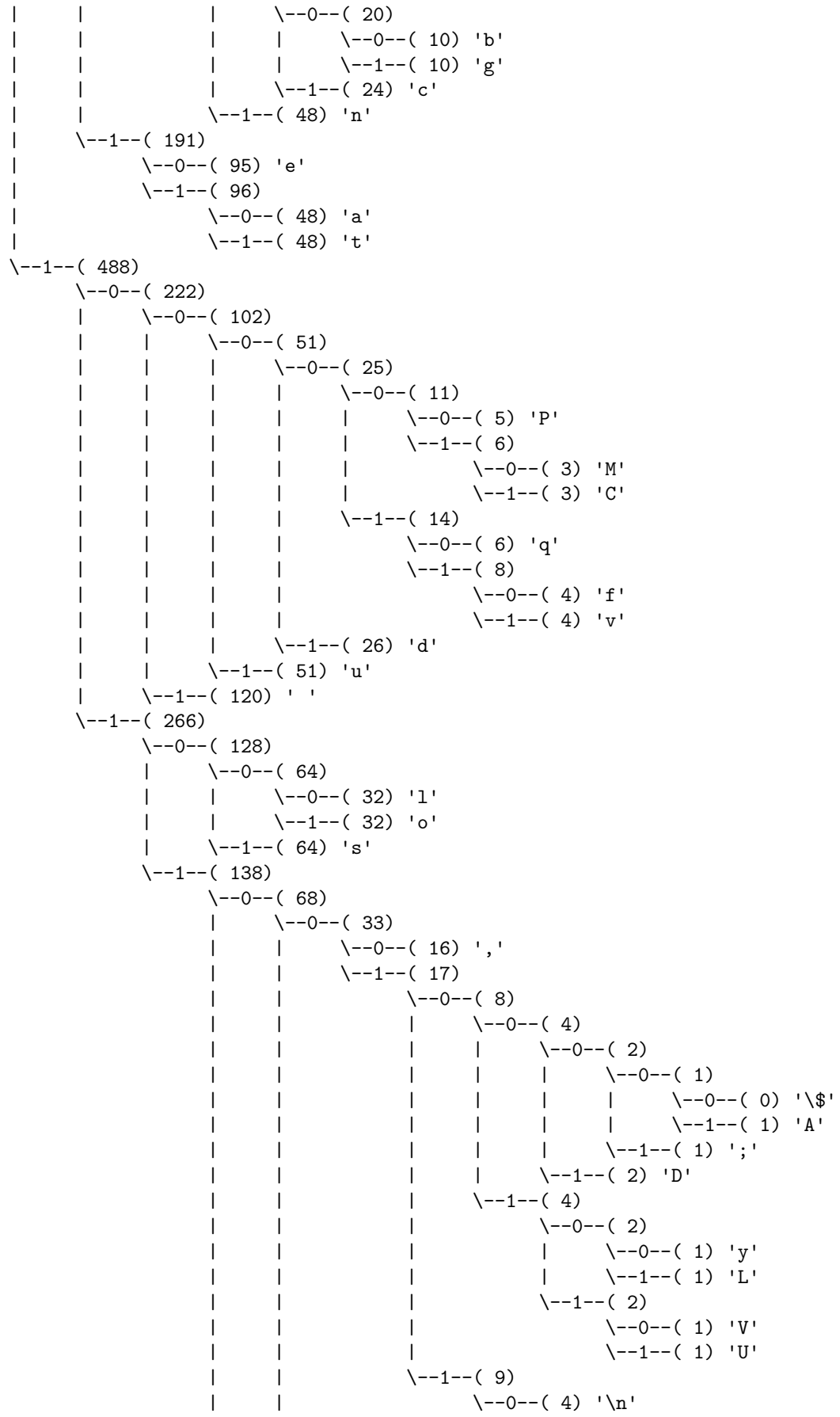
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

Il est mis là pour la satisfaction de voir l'application fonctionner...

4.7.1 Résultat (sans *xxd*, ce serait trop long)

Pour `Compression`, le mode *bavard* donne:

```
( 848)
\--0--( 360)
|     \--0--( 169)
|     |     \--0--( 77)
|     |     |     \--0--( 37) 'm'
|     |     |     \--1--( 40) 'r'
|     |     \--1--( 92)
|     |           \--0--( 44)
```


\--0--(20)
\--0--(10) 'b'
\--1--(10) 'g'
\--1--(24) 'c'
\--1--(48) 'n'
\--1--(191)
\--0--(95) 'e'
\--1--(96)
\--0--(48) 'a'
\--1--(48) 't'
\--1--(488)
\--0--(222)
\--0--(102)
\--0--(51)
\--0--(25)
\--0--(11)
\--0--(5) 'P'
\--1--(6)
\--0--(3) 'M'
\--1--(3) 'C'
\--1--(14)
\--0--(6) 'q'
\--1--(8)
\--0--(4) 'f'
\--1--(4) 'v'
\--1--(26) 'd'
\--1--(51) 'u'
\--1--(120) ' '
\--1--(266)
\--0--(128)
\--0--(64)
\--0--(32) 'l'
\--1--(32) 'o'
\--1--(64) 's'
\--1--(138)
\--0--(68)
\--0--(33)
\--0--(16) ','
\--1--(17)
\--0--(8)
\--0--(4)
\--0--(2)
\--0--(1)
\--0--(0) '\$'
\--1--(1) 'A'
\--1--(1) ';'
\--1--(2) 'D'
\--1--(4)
\--0--(2)
\--0--(1) 'y'
\--1--(1) 'L'
\--1--(2)
\--0--(1) 'V'
\--1--(1) 'U'
\--1--(9)
\--0--(4) '\n'

```
                |     |                \--1--( 5)
                |     |                      \--0--( 2) 'S'
                |     |                      \--1--( 3) 'h'
                |     \--1--( 35)
                |           \--0--( 17) '.'
                |           \--1--( 18) 'p'
                \--1--( 70) 'i'
```

```
'm'  --> 0000
'r'  --> 0001
'b'  --> 001000
'g'  --> 001001
'c'  --> 00101
'n'  --> 0011
'e'  --> 010
'a'  --> 0110
't'  --> 0111
'P'  --> 1000000
'M'  --> 10000010
'C'  --> 10000011
'q'  --> 1000010
'f'  --> 10000110
'v'  --> 10000111
'd'  --> 10001
'u'  --> 1001
' '  --> 101
'l'  --> 11000
'o'  --> 11001
's'  --> 1101
','  --> 111000
'\$' --> 11100100000
'A'  --> 11100100001
';'  --> 1110010001
'D'  --> 111001001
'y'  --> 1110010100
'L'  --> 1110010101
'V'  --> 1110010110
'U'  --> 1110010111
'\n' --> 11100110
'S'  --> 111001110
'h'  --> 111001111
'.'  --> 111010
'p'  --> 111011
'i'  --> 1111
```

Pour faire bref (et raccourcir le rapport), `Decompression` renvoie bien le même arbre et le même texte. Le test est donc validé et il semble à ce point que notre programme fonctionne plutôt bien.

5 Organisation du travail

S'organiser a été une tâche plutôt difficile au début du projet. Le tableau ci-dessous tente cependant de détailler le rôle de chacun:

5.1 Tableau `Qui (a) fait Quoi`

YR : Yan RABEFANIRAKA
AF : Abel FARES

Quoi ?		Qui ?			
Modules	Sous-programmes	Spécification	Codage	Test	Relecture
arbre	AB_Initialiser	YR	YR	YR	AF
	AB_Detruire	YR	YR	YR	AF
	AB_Est_Vide	YR	YR	YR	AF
	AB_Est_Feuille	YR	YR	AF	AF
	Assigner	AF	AF	YR et AF	YR
	AB_Profondeur	AF	AF	AF	YR
	AB_egaux	YR	YR	YR	AF
	AB_Creer_Noeud	AF	YR	YR	AF
	AB_Assigner_Cle	YR	YR	YR	AF
	AB_La_Valeur	YR	YR	YR	AF
	AB_La_Cle	YR	YR	YR	AF
	AB_Fils_Gauche	YR	YR	YR	AF
	AB_Fils_Droit	YR	YR	YR	AF
	AB_Nombre_Noeuds	AF	YR	YR	AF
	AB_Afficher_Debug	YR	AF et YR	YR	YR
	Generer_Arbre_Parcours	YR	YR	YR	AF
	Decoder_Symbole_Compresse	YR	YR	YR	AF
	Est_vide_tab_arbre	AF	AF	AF	YR
	Arbre_Present	AF	AF	AF	YR
	Taille_Table_Arbre	AF	AF	AF	YR
	Arbre_indice	AF	AF	AF	YR
	Cle_Arbre_indice	AF	AF	AF	YR
	Valeur_Arbre_indice	AF	AF	AF	YR
	Enregistrer_Arbre	AF	AF	AF	YR
	Detruire_tab_arbre	AF	AF	AF	YR
	Detruire_Cellule_Arbre	AF	AF	AF	YR
	Mins_Tab_Arbre	AF	AF	AF	YR
	Table_symbole_infixe	AF	AF	AF	YR
	Afficher_Tab_Arbre	AF	AF	AF	YR
	Init_infixe	AF	AF	AF	YR
	Enregistrer_infixe	AF	AF	AF	YR
	Infixe_Indice	AF	AF	AF	YR
	Detruire_infixe	AF	AF	AF	YR
	Taille_infixe	AF	AF	AF	YR
	Enregistrer_infixe_indice	AF	AF	AF	YR
	Assigner_Infixe	AF	AF	AF	YR
	Symbole_encodage	AF	AF	AF	YR
	Init_symbole	AF	AF	AF	YR
	Enregistrer_symbole	AF	AF	AF	YR
	Symbole_Indice	AF	AF	AF	YR
	Taille_symbole	AF	AF	AF	YR
	Detruire_symbole	AF	AF	AF	YR
	Enregistrer_Tab_Huffman	AF	AF	AF	YR
	Encodage_symbole	AF	AF	AF	YR
	Detruire_Tab_Huffman	AF	AF	AF	YR
	Trouver_symbole_fin	AF	AF	AF	YR
	Afficher_infixe	AF	AF	AF	YR
	Afficher_symbole	AF	AF	AF	YR
octet	Detruire_Tab_Octet	YR	YR	YR	AF
	Est_Meme_Octet	YR	YR	YR	AF
	Lire_Fichier_Bin	YR	YR	YR	AF

Quoi ?		Qui ?			
compression	Different_fin	AF	AF	AF	YR
	Inferieur_Strict	AF	AF	AF	YR
	To_Character	AF	AF	AF	YR
	To_T_String	AF	AF	AF	YR
	AB_Afficher_Cle	AF	AF	AF	YR
	AB_Afficher_Donnee	AF	AF	AF	YR
	AB_Afficher	AF	AF	AF	YR
	Min	AF	AF	AF	YR
	Indice_symbole_fin	AF	AF	AF	YR
	Shift_Left	AF	AF	AF	YR
	Gestion d'arguments	AF	AF et YR	YR	YR
	Tableau de fréquences	AF	AF	AF	YR
	Arbre de Huffman	AF	AF	AF	YR
	Afficher Table de HF	AF	AF	AF	YR
	Écrire dans le fichier	AF	AF	AF	YR
decompression	Est_Hff	YR	YR	YR	AF
	Different_fin	YR	YR	YR	AF
	To_T_String	AF	AF	YR	YR
	To_UB_String	YR	YR	YR	AF
	Gestion d'arguments	YR	YR	YR	AF
	Récupérer symboles	YR	YR	YR	AF
	Récupérer arbre de HF	YR	YR	YR	AF
	Décoder texte	YR	YR	YR	AF

Il est assez clair que, bien que ça n'ait pas été notre volonté initiale, `Compresser` a principalement été réalisé par Abel, là où `Décompresser` a été principalement fait par Yan. Cette division très inégale des tâches est abordée plus en large dans la section suivante, mais vis-à-vis de l'organisation, elle est certainement causée par un manque de communication au préalable sur les sous-programmes que chacun doit réaliser. Elle semble aussi être partiellement causée par la superficialité du raffinage d'origine, qui nous a mené à improviser pleins de nouvelles structures de données/sous-programmes à la volée, et qui "oblige" notamment le programmeur ayant la main – qui a besoin de ces nouveautés – à *monopoliser* le répertoire pour les ajouter.

6 Difficultés rencontrées

Cette section n'a pas énormément changé depuis le précédent rapport.

6.1 Choix des structures de données

Trouver les structures de données les plus adaptées à notre problème n'a pas été aisé. On a souvent été tenté de prendre les structures les plus "puissantes" en premier lieu, comme une table de hachage par exemple. Or, cette dernière ne sert à rien puisqu' on travaille sur des symboles. Il en va de même pour les implémenter ou les modifier selon nos besoins. On pense pour instance à la fonction qui renvoie l'encodage Huffman d'un symbole à partir de l'arbre de Huffman. Nous avons finalement opté pour une solution facile : créer un nouveau type de donnée pour chaque partie principale du problème. Nous aurions pu factoriser toutes ces structures de données dans une structure complexe mais ça aurait coûté beaucoup de temps et d'abstraction. Notre solution permet donc d'avoir des structures aux objectifs clairs et simples et permet de compartimenter le problème; son désavantage est la prolifération des sous-programmes associés à chaque structures de données.

6.2 Lier les parties du sujet ensemble

Certaines parties du sujet manquent de transition ou sont (volontairement ?) détaillées différemment. Là où certaines sont très claires, d'autres le sont beaucoup moins. Cela nous rajoute un challenge de compréhension supplémentaire qui fait certainement partie de l'exercice.

6.3 Manipuler la modularité de Ada et savoir où et quand implémenter/instancier des structures de données.

Malgré l'expérience des TPs et projets précédents, on se retrouve encore parfois à hésiter sur l'endroit où initialiser un package générique, à savoir quoi mettre en privé ou non, etc. Cela se règle cependant tout seul au fil du projet. L'utilisation de la généricité a été cependant assez modérée: En effet, certaines structures de données auraient pu être dérivées de précédentes. On aurait même pu imbriquer les deux packages LCA et ARBRE mais pas sans de nombreux soucis surtout pour faire correspondre les bons types de données entre les 3 fichiers, et une certaine inefficacité au niveau des outils à disposition nous forçant probablement à développer beaucoup plus de code dans le fichier compression, ce qui n'est pas l'objectif quand on utilise des packages.

6.4 Gérer notre temps et le répertoire

En effet, un BE d'automatique étant à finaliser pour le même week-end que le premier rendu de `Compression`, la gestion du temps de travail en équipe a été quelque peu médiocre et peu *agile*. On peut remarquer une période où un membre fait beaucoup sur le projet pendant que l'autre travaille sur le BE puis inversement, résultant en un déséquilibre sur la compréhension de l'avancement de l'autre (la division des tâches en modules et procédures s'est vite arrêtée) et sur le travail fourni en lui-même. Sur les derniers jours, le déséquilibre était tel qu'un membre s'est retrouvé à terminer entièrement le BE tandis que l'autre a finalisé à lui tout seul La compression.

Ainsi, on a régulièrement rencontré des conflits de répertoire à cause de ce problème, puisque quand un membre se lance sur sa partie du code tandis que l'autre est encore dessus depuis plusieurs heures car la sienne en dépend, cela limite grandement la possibilité de travailler en même temps.

7 Solutions

- La plupart des solutions au problème de code se sont montrées à nous d'elles-même, soit en continuant de creuser le sujet, soit en demandant à l'un des professeurs responsables du projet, notamment lors du temps de travail au sein de l'école. La solution la plus simple a parfois été choisie peut-être en dépit d'une solution plus optimale.
- Pour ce qui est de l'organisation du travail, elle était tellement brouillonne en premier lieu que le plus gros du travail *i.e* avoir les deux modules principaux fonctionnels a au final été étrangement terminé avant la rentrée, mais ce parce que chacun garde individuellement la main sur son programme jusqu'à le finir au complet, quitte à veiller des heures la nuit. Cela ne semble évidemment pas être un fonctionnement sain à long terme, mais il semblerait que cela ait fonctionné pour nous deux au final.

8 État actuel du projet

8.1 Avancée de Compression

Au moment de l'écriture de ce rapport, `Compression` fonctionne comme attendu, pour des fichiers texte seulement.

8.1.1 Valgrind

Aucun problème de mémoire n'est détecté par Valgrind (exemple avec `texte_long.txt`):

```
==1223280==
==1223280== HEAP SUMMARY:
==1223280==     in use at exit: 65,088 bytes in 4,068 blocks
==1223280==   total heap usage: 5,739 allocs, 1,671 frees, 143,192 bytes allocated
==1223280==
==1223280== LEAK SUMMARY:
==1223280==    definitely lost: 14,816 bytes in 926 blocks
==1223280==    indirectly lost: 50,272 bytes in 3,142 blocks
==1223280==      possibly lost: 0 bytes in 0 blocks
==1223280==    still reachable: 0 bytes in 0 blocks
==1223280==         suppressed: 0 bytes in 0 blocks
```

```
==1223280== Rerun with --leak-check=full to see details of leaked memory
==1223280==
==1223280== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

On retiendra quand même que tous les "*heap*" blocks ne sont pas libérés; Il existe peut-être des scénarios où il y aura débordement.

8.2 Avancée de Décompression

Tout comme `Commpression`, `Decompression` fonctionne correctement. Il est écrit des fichiers texte au lieu de fichiers binaires, mais cela peut être changé de manière relativement simple pour les améliorations possibles de l'application.

8.2.1 Valgrind

Aucun problème de mémoire ici non plus:

```
==1223758==
==1223758== HEAP SUMMARY:
==1223758==     in use at exit: 0 bytes in 0 blocks
==1223758==   total heap usage: 1,811 allocs, 1,811 frees, 437,712 bytes allocated
==1223758==
==1223758== All heap blocks were freed -- no leaks are possible
==1223758==
==1223758== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Ici, tous les "*heap blocks*" sont libérés.

8.3 Insuffisance(s)

Il s'agit des endroits (gestion d'équipe *ET* code) où un réel manquement peut être observé et où une amélioration semble nécessaire.

8.3.1 Division des tâches

La division brouillonne du travail à faire nous a mené à différents problèmes notamment vis-à-vis de l'interprétation/l'implémentation de certains pans du sujet. Malgré l'accomplissement de l'objectif principal du projet, on aurait beaucoup bénéficié (en particulier sur le plan physique avec des nuits de sommeil sacrifiées) à mieux se répartir les tâches. Un format possible serait celui de se répartir un nombre fixe de sous-programmes à effectuer par personne et par semaine selon les besoins du programme principal. En clair, laisser une unique personne au commande du répertoire (avancer dans le programme principal ET les sous-programmes qu'il utilise) peut vite s'avérer risqué car on est très vite tenté de vouloir finir notre "partie" et d'au final en faire trop (sans compter le blocage de certains modules qui sont en cours de modification parce que les sections à ne pas modifier pour empêcher les erreurs de *merging* non pas été communiquées au préalable).

8.3.2 Fichiers binaires

Comme dit plusieurs fois précédemment, notre difficulté à nous organiser correctement au début du projet nous a mené à ne pas implémenter une partie du cahier des charges concernant la lecture/écriture de fichiers: Le sujet nous demande de ne lire et n'écrire que des fichiers binaires, ce que nous ne faisons pas puisque `Compression` lit des fichiers texte codés en ASCII et `Decompression` écrit un fichier texte malgré l'extension `hff.d`. Cette erreur est due au manque de visibilité sur les avancées de chacun comme nous étions mutuellement plongés dans ce que nous faisions. Même en relisant le code de l'autre, ces erreurs n'ont pas été descellées avant le 1er rapport où on y fait tout de même mention. Alors, avec en premier lieu la volonté de compléter l'objectif premier de chaque module (*i.e* compresser et décompresser à l'aide du codage de Huffman) et avec du temp limité, on a préféré reléguer cette caractéristique au statut de dernières retouches.

8.4 Pistes d'amélioration

- Les structures de données utilisées dans `Compression` et définies dans `Arbre` sont quelque peu répétitives et bénéficieraient à être regroupées en une seule.

- Le tableau de symbole utilisé dans `Decompression` pourrait judicieusement être remplacé par une LCA, ce qui faciliterait aussi la correction de la sortie en fichier binaire.
- Un erreur liée à un problème d'index peut très marginalement apparaître mais n'empêche pas le décodage du texte complet. Pour autant, sa gestion en programmation défensive approximative laisse à désirer et pourrait être améliorée.
- Un octet est arbitrairement rempli après l'insertion du parcours infixe dans `Compression`. Cela n'est pas nécessaire puisque `Decompression` sait quand le parcours est terminé.

9 Bilans personnels et conclusion

Abel *(14/12/2024)* Mon travail sur la spécification ayant été assez léger je suis arrivé sur le projet avec une compréhension trop surfacique des différents processus impliqués dans le problème. J'ai donc appris au cours du projet faisant beaucoup d'erreurs et devant changer mes méthodes plusieurs fois et j'ai aussi dû apprendre à éviter de gâcher du temps à chercher la solution optimale dans certains cas. La compréhension du sujet m'a cependant pris un temps assez remarquable notamment celle de l'exemple du sujet qui lit apparement les caractères du texte dans un ordre arbitraire et construit son arbre en conséquences, il m'est donc impossible de comparer directement mon rendu et celui du sujet, mais cet exemple permet cependant de vérifier la compréhension des concepts. Malgré le temps important passé sur le projet (quasiment 25 heures) il y a toujours un problème de détection de certains caractères "line feed" **\n** et les sous-programmes de nettoyage des pointeurs ne sont toujours pas au point ce qui est d'une grande frustration pour moi. Cependant ce projet m'as appris beaucoup sur la généricité en Ada, la gestion de types string et characters ainsi que les modules `Ada.Command_Line` et `Ada.Streams_IO`.

(11/01/2025) J'ai pu remédier à tous les problèmes concernant les écarts du code de compression par rapport aux attendus notamment les problèmes de détection de certains caractères et de fuite de mémoire. Cependant j'ai décidé de ne pas implémenter de structure de données plus optimale car cela aurait impliqué de réécrire une bonne partie des procédures et surtout une utilisation pénible de la généricité à travers plusieurs modules. J'ai conclu que malgré les améliorations encore possibles ce projet m'a permis de comprendre et d'utiliser pleinement les enseignements des cours et m'a offert une compréhension plus globale du langage de programmation qu'est Ada.

Yan *(14/12/2024)* Honnêtement, le temps de travail a été assez inégal sur cette semaine de travail; J'ai d'abord spécifié et implémenté nos structures de données pour préparer la compression, puis Abel a très vite avancé sur la compression. Ayant le BE d'automatique à rendre pour nous deux, je m'y suis attelé sérieusement tout seul pour laisser à mon partenaire le temps d'avancer sur le projet. Mon investissement personnel a surtout été de revenir régulièrement sur le code pour commenter ce que proposait Abel, corriger des erreurs ou clarifier des points du sujet que j'ai assez bien dévoré. Pourtant, cela est assez frustrant car, malgré le temps que prenait notre autre travail de groupe, j'aurais aimé pouvoir mieux répartir la charge pour mon partenaire. Cela comprend aussi les problèmes de répertoire rencontrés, ce qui m'a parfois obligé à "discard" des modifications de mon côté ou à attendre que mon partenaire finisse certaines procédures/ certains modules.

(11/01/2025) En définitive, ce projet a été plutôt enrichissant concernant la gestion d'un projet en binôme sur une période chargée; Il nous a obligé à déléguer – certes pas de manière optimale – la charge de travail à chacun des membres, et à respecter les *deadlines* notamment. Beaucoup de leçons sont à tirer sur la manière de se répartir ces tâches comme cité précédemment. De même, c'était une bonne application de la programmation modulaire qui m'a obligé à utiliser tout ce qui a été vu en cours voire un peu plus. Ensuite, la satisfaction de voir l'application marcher (presque) comme attendu soulage la quantité de temps assez conséquente allouée à la réaliser. Pour autant, notre/ma gestion discutable du temps de travail sur le projet m'a par exemple mené à une nuit blanche quelques jours avant la rentrée pour être sûr de finaliser le module `Decompression`, dont l'impact s'est largement ressenti les jours suivants sur ma santé physique puisque tout mon cycle de sommeil est depuis perturbé et peine à se remettre en place. C'est donc une leçon importante que je n'oublierai pas vis-à-vis du débordement du temps de travail sur le temps de repos et des prescriptions à mettre en place pour éviter d'en arriver là. La preuve notamment avec ce rapport que j'écris maintenant depuis une semaine à l'aide de mon binôme.